

CSCI 1430 Final Project Report:

PerfectingPoker: Spotting Winning Hands with CV

Team names: Rio Voss-Kernan, Jad Alkarim Al Smail, and Yassir Aouf

TA name: John Farrell. Brown University

Abstract

PerfectingPoker is a real-time computer vision-based poker assistant designed to identify playing cards from live camera input and provide strategic insights using probability simulations. The system detects and isolates cards by identifying contours and applying size thresholds. Each detected card is cropped to extract the top-left corner containing the rank and suit, which are further segmented into separate images. These are then classified using two trained Convolutional Neural Networks (CNNs): one for rank and one for suit recognition. The classified cards are annotated on-screen, and the program continuously runs 200 Monte Carlo simulations to compute the probabilities of possible poker hands (e.g., pair, two pair, flush, straight, etc.). These probabilities are displayed in real-time on the camera feed. The system updates as the cards on the table change, maintaining accurate recognition and analysis. PerfectingPoker showcases the integration of computer vision and probabilistic modeling to enhance decision-making in poker gameplay. PerfectingPoker is intended as a training tool for face-up card scenarios to help players practice and develop intuition for poker probabilities.

1. Introduction

Understanding hand probabilities is a fundamental part of mastering poker strategy, yet even experienced players can struggle to accurately estimate odds during play. Calculating the likelihood of forming various hands—such as a flush, straight, or full house—requires quick mental math, awareness of visible cards, and a solid grasp of combinatorics. While such skills are critical to improving decision-making at the table, they are difficult to develop without extensive experience or external tools for feedback.

PerfectingPoker is a computer vision-powered training assistant designed to help poker players practice and build intuition for hand probabilities in a visual and interactive way. Rather than being a tool for use during real games, the system is intended for controlled environments where all cards are placed face-up, enabling users to learn in a trans-

parent and instructive setting. This focus also addresses a key limitation of our system—accurate detection and classification is currently only feasible with visible card faces, which would not be available in a typical competitive poker scenario.

By providing immediate probabilistic feedback in response to different card configurations, PerfectingPoker serves as an educational tool that helps players deepen their understanding of poker odds and decision-making. It bridges the gap between theory and practice, making statistical reasoning more accessible through real-time visual interaction.

2. Related Work

Our application pulls from several existing infrastructures. All neural networks were trained and modeled using the TensorFlow deep-learning framework, and OpenCV's library provided many necessary image processing functions. Our image processing approach of contour finding and rank/suit isolation also pulls inspiration from the similar work done by Edje Electronics [1].

3. Method

Our approach the the problem can be broken down into three main tasks: finding the cards, classifying the cards, and simulating the cards. To set up for finding the cards, we rely upon an aerial view video feed of the gameplay area. From this live feed, we extract frames to be processed, classified, and represented on the UI. This image processing pipeline is outlined in Figure 1.

To find the cards within the frame, a thresholded contour-finding algorithm is used to extract the outlines of each card. These outlines are converted into polygons, from which corners can be extracted for card alignment. Identifying the corners allows for a perspective warp from the skewed quadrilateral to a cropped and properly aligned top-down view of the card. This newly extracted card image will make the model invariant to background, perspective alignment, and minor rotations.

Once the cards are extracted into isolated and aligned images, we can crop the image into separate isolated suit

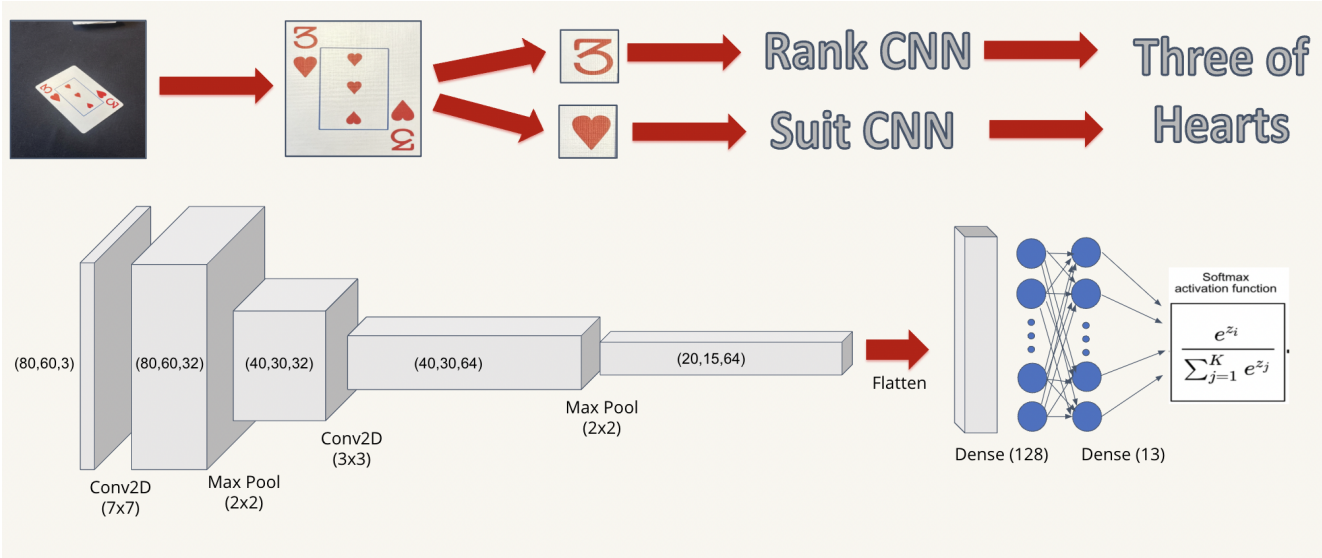


Figure 1. Image processing pipeline and rank/suit CNN architecture.

and rank images. These cropped images are then passed to two trained Convolutional Neural Networks (CNN), which accurately classify the rank (e.g., 5, Queen) and suit (e.g., Clubs, Diamonds) of each card. These CNNs were trained on manually captured top-down images of the 52 cards in a Bicycle Jumbo set of playing cards, which underwent the same preprocessing described above. Both CNNs use the same model architecture which can be seen in Figure 1.

Once all cards in the frame are identified, the system performs backend calculations to estimate the probabilities of achieving various poker hands, such as a pair, two pair, straight, flush, etc., given the current state of the game. To achieve this efficiently, the system runs hundreds of Monte Carlo simulations, simulating possible future card draws and calculating the frequency of each hand type to approximate their probabilities. These three steps of identification, classification, and simulation provide the user with their desired hand stats for each captured frame.

4. Results

Our final product (Figure 2) demonstrates the successful integration of card classification and probability analysis into a single real-time interface. Cards placed on a desk are correctly identified by the computer vision model, with each prediction visually overlaid on the card itself in the frame. Detected cards are then passed into a Monte Carlo simulation engine, which generates and displays updated probabilities for achieving various poker outcomes (e.g., flush, straight, full house). These results appear on the left side of the interface, offering the player immediate, real-time statistical feedback as gameplay progresses.

Our CNN was trained on a custom dataset consisting of



Figure 2. Frame captured from final product UI: classifying cards on the right and providing probability statistics on the left.

manually captured images of each card in a specific deck. By using this tailored dataset, we ensured consistency in card appearance (e.g., lighting, perspective, noise), which significantly improved classification performance. The final model achieved 98.7 percent accuracy on a validation set of held-out images from the same deck, correctly predicting both rank and suit in nearly every instance. Figure 3 shows the training and validation accuracy as the number of epochs grows, illustrating the steady convergence of the model.

4.1. Technical Discussion

Our final implementation has several interesting tradeoffs. Firstly, the model relies upon identifying the border outline of cards to crop and warp them during preprocessing. This strategy allows our model to be invariant to background, perspective, and rotation up to 45 degrees in either direc-

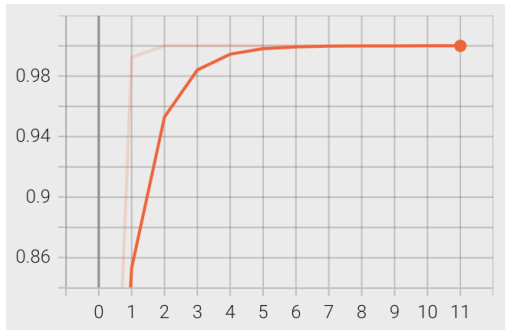


Figure 3. Training and validation accuracy across epochs for the final CNN model.

tion. This is because the system identifies the card corners for the alignment warp to crop and rotate the card, but an over-rotated card will be aligned in the wrong orientation. Additionally, this strategy allows us to identify and classify any number of cards in any position within the frame by identifying the borders of each. Whereas, with a traditional CNN model trained on single cards, the system would not have been able to work with cards in parallel.

However, this has its downsides. Because the model relies upon card outlines, the traditional way of holding cards overlapping in one's hand would be impossible for the model to identify. Therefore, a real user would need to lay out their cards on the table or hold them apart from each other in the camera's vision. This is not a critical issue, however, as the product is generally meant for single-player practice sessions rather than real-time game play, so face-up play is acceptable.

Secondly, our CNN models were trained on very limited data. The inputs to these models will be very specifically aligned and cropped card segments, so overfitting to the training data is only an issue if the deck of cards changes. Currently, our model is trained and preprocessed specifically for a Jumbo Bicycle deck of cards, and would not work effectively on other decks. To overcome this, we would need a larger dataset of more decks of cards and to make our rank/suit segmentation less strict to account for the differences in icon positions across decks.

Lastly, our model uses Monte-Carlo simulations to provide probabilities for hitting each poker hand. This is technically non-optimal. There is a ground truth probability for hitting each hand from each state; however, calculating these ground truths becomes a problem of large-scale probability calculations, as each card can contribute to many hands in many different ways. This was outside the scope of the project and not directly related to the computer vision we sought to solve, so we opted to use Monte-Carlo simulations instead, which provide a good estimation of the ground truths without the computational complexity.

5. Conclusion

In this project, we developed PerfectingPoker, a real-time poker assistant that combines traditional computer vision techniques, convolutional neural networks, and Monte Carlo simulations to deliver live predictions of poker hand probabilities. From detecting cards via detecting contours, to isolating and classifying their suit and rank using two specialized neural networks, our system translates a video feed into a list of poker cards. It then computes updated probabilities for all common poker outcomes—flush, straight, full house, and more—every time a new card appears. The result is a fast, responsive tool that helps players understand the mathematical expectations of the game without interrupting its natural flow.

Beyond its technical implementation, PerfectingPoker demonstrates how real-time AI feedback can enhance gameplay in meaningful ways. For beginners, it serves as an educational tool, helping them internalize the rules of poker and recognize patterns through experience rather than memorization. For more experienced players, it becomes a performance enhancer, offering immediate feedback and enabling users to compare their instincts with statistically sound expectations. By doing so, the tool bridges a gap in many poker training resources: it provides real-time, continuous, and non-intrusive support while preserving the social and strategic aspects of the game.

The system performed well in our testing conditions, successfully detecting and classifying cards with high accuracy and maintaining low-latency updates of the probability engine. However, there are several directions for future work. First, while our neural networks handled most standard card formats, classification performance could be improved by expanding the training set to include cards with more stylistic variation, different sizes, or atypical fonts. Similarly, although our pipeline handled slight overlaps and lighting changes reasonably well, a more robust card detection system could improve performance in less controlled environments. Additionally, supporting partially hidden or handheld cards remains a challenge to our implementation. On the user experience side, we envision additional functionality such as opponent card tracking, support for different game variants (e.g., Texas Hold'em vs. Five Card Draw), and augmented reality integration for hands-free assistance. The model could also be extended to give players basic strategic suggestions or flag statistically risky moves during play, creating an even more comprehensive practice environment.

Overall, PerfectingPoker showcases the power of combining classical image processing, deep learning, and probabilistic modeling into a real-time application with both

practical and educational value. It not only supports users in understanding the game more deeply but opens the door to building intelligent assistants for other card-based or turn-based games where prediction, pattern recognition, and live feedback are beneficial.

References

- [1] Edge Electronics. Playing card detection using opencv-python on the raspberry pi. *Youtube*, 2018. [1](#)

Appendix

Team contributions

Rio Voss-Kernan I worked largely on the image classification task. I tried using a variety of datasets and CNN architectures to classify cards effectively while generalizing them to real-world uses. I implemented a residual block CNN architecture and data preprocessing pipeline for the original strategy, as well as tried a pretrained ResNet, neither of which ended up generalizing well. Then, I redesigned the architecture for our final model with the new preprocessing strategy and trained the model on manually captured images of playing cards on my desk. Lastly, I implemented the card contour identification and preprocessing to be compatible with the new CNNs for final classification.

Jad Alkarim Al Smail I was working on the card detection component of the system, ensuring reliable performance under varying lighting conditions and card orientations. I implemented the technique to extract and crop the top-left corner of each detected card, segmenting the rank and suit regions for classification. Initially, I developed a feature-matching classification method that compared these cropped regions with reference images, but we later transitioned to using two trained Convolutional Neural Networks (CNNs) for improved accuracy. I tested both approaches to ensure correctness and robustness. Additionally, I implemented real-time camera frame capture using OpenCV with threading to allow efficient, continuous processing of each frame.

Yassir Aouf I was responsible for developing the probabilistic analysis system and integrating it into the overall pipeline. I implemented the Monte Carlo simulation engine that estimates the probability of achieving various poker hands (such as flushes, straights, or full houses) based on the cards currently detected by the model. This involved modeling the remaining deck, simulating thousands of random draws, and computing the outcomes to produce statistically meaningful probabilities. I then integrated this simulation logic into the computer vision pipeline so that detected cards would trigger real-time updates of the hand probabilities directly on

the video feed. In addition to this, I was actively involved in debugging and refining multiple components of our project throughout development—particularly when testing the effectiveness of different card classification approaches and helping identify issues during model integration and testing.

Things We Tried That Didn't Work

We tried several strategies for card classification before landing on our final model. Firstly, we tried training a general CNN to classify across 52 classes with a relatively small online dataset we found. This task did not perform well in training or testing due to the difficulty of the task and the variance in the dataset. We then tried to use the same system with a new dataset of 10,000 images of one deck of cards, assuming that this would learn much better. However, we faced a problem where the model would not generalize to real images despite high testing accuracy and seemingly little difference between real images and the input data. This led us to separate our model into two different CNNs— one for rank and one for suit. From here, the system improved slightly, but we realized that it would be much more robust if we could extract the card from the background first.

Before training the CNNs for the rank and the suit, we tried an approach in which we extract the corners then compare them to reference images and match the cards based on the number of similar pixels, but then we figured that training CNNs would provide more accuracy in classification.

5.1. SRC critique

We appreciate the thoughtful critique and the important ethical, social, and technical concerns it raises. Many of the points made align with conversations we've had ourselves during development, and we recognize that any technology—particularly one involving games of chance and skill—requires careful consideration of its impact on users and the broader ecosystem.

First, regarding the concern that our poker AI undermines fair play and player autonomy: this is a valid and nuanced issue. Our project's goal was not to replace human decision-making but rather to enhance understanding and provide a tool for learning and analysis. While it is true that real-time probability calculation can offer a competitive advantage, we see this technology primarily as a training aid or a post-game analysis tool rather than something to be used during high-stakes competitive play. We acknowledge that unfettered use of such tools in real games could disrupt fairness, and

we would support usage boundaries (e.g., banning real-time use in tournament settings) to preserve integrity. That said, we believe that democratizing access to strategy tools can also level the playing field for beginners who wouldn't otherwise have the resources to develop competitive intuition.

As for the critique that our proposal assumes a high level of background knowledge and lacks support for beginners: this is excellent feedback. Our initial focus was on the technical side of the implementation, but we realize that, to make the tool accessible, we need to better communicate not only what the optimal decisions are but also why they are optimal. We plan to revise the interface and supporting documentation to include more explanations, tutorials, and visual guides, which would help new players understand fundamental concepts of probability, expected value, and bluffing. In this way, our project could evolve into a more inclusive learning resource rather than something only for advanced users.

In sum, we are grateful for the critique and believe it provides a valuable perspective on how to evolve our project responsibly. We remain excited about the potential of our tool to educate, support learning, and deepen appreciation for the complexities of poker, while also committing to safeguarding values such as fairness, transparency, and privacy.